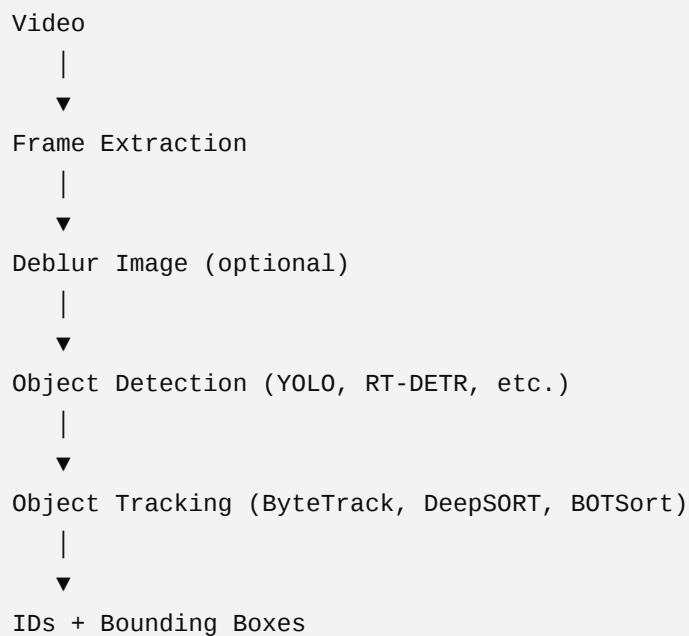


[↑ Share](#) ...

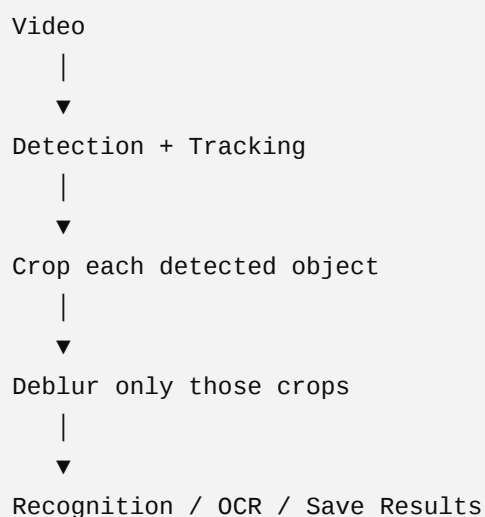
so i've built this ml model where i taught my model to detect cars and objects and now their id's are shown and now i want to do deblurring in my model so how to do it

If your model already performs **object detection + tracking** (so each car/person/object has a persistent ID), then adding **deblurring** is usually done as a **separate image restoration step**, not inside the detector itself.

The typical pipeline is:



or, if you only want to improve the detected objects:



The second approach is much faster.

Step 1: Determine what type of blur you have

Different models work best for different blur types.

Blur Type	Cause	Best Method
Motion Blur	Moving vehicle/camera	Motion Deblurring
Defocus Blur	Camera out of focus	Defocus Deblurring
Low Resolution	Distant objects	Super Resolution
Rain/Fog Blur	Weather	Image Restoration
Noise	Low light	Denoising

For moving cars, **motion blur** is the most common.

Step 2: Choose a Deblurring Model

Some popular deep-learning models are:

1. MIMO-UNet (Recommended)

Very fast.

Input

Blurred Image



Output

Sharp Image



Advantages

- Real-time capable
- High PSNR
- Lightweight

2. Restormer

One of the strongest general image restoration models.

Advantages

- Excellent quality
 - Handles motion blur
 - Handles noise
 - Handles rain
-

3. NAFNet

Very fast.

Good for deployment.

4. DeblurGAN-v2

GAN-based.

Advantages

- Preserves details
 - Good-looking images
-

5. HINet

Excellent restoration quality.

Step 3: Add it to your Detection Pipeline

Suppose you use YOLO.

Instead of

</> Python



▶ Run

Frame



YOLO

Do

Python



Run

Frame



Deblur Network



YOLO

Pseudo-code:

Python



Run

```
frame = camera.read()

sharp = deblur_model(frame)

results = yolo(sharp)

tracker.update(results)
```

Step 4: Deblur Only Detected Objects (Recommended)

Suppose YOLO detects

Car

bbox = [100, 200, 300, 450]

ID = 17



Crop

Python



Run

```
car = frame[y1:y2, x1:x2]
```

Run

Python



Run

```
sharp_car = deblur(car)
```

Replace

</> Python



▶ Run

```
frame[y1:y2, x1:x2] = sharp_car
```

Advantages

- Much faster
- Deblur only vehicles
- Lower GPU usage

Step 5: Example Pipeline

Video



↓

YOLO Detection

↓

ByteTrack

↓

For every object:

 Crop

 ↓

 Deblur

 ↓

Display

Step 6: Using PyTorch

</> Python



▶ Run

```
from basicsr.archs.restormer_arch import Restormer

model = Restormer()

model.load_state_dict(torch.load("restormer.pth"))

model.eval()
```

Then

</> Python



▶ Run

```
with torch.no_grad():
    output = model(image_tensor)
```

Convert back

</> Python



▶ Run

```
output = output.squeeze().permute(1,2,0).cpu().numpy()
```

Step 7: OpenCV Integration

</> Python



▶ Run

```
frame = cap.read()

results = detector(frame)

for box in results:

    x1,y1,x2,y2 = box

    crop = frame[y1:y2,x1:x2]

    sharp = deblur(crop)

    frame[y1:y2,x1:x2] = sharp
```

Step 8: If You Need Real-Time Speed

Approximate GPU requirements:

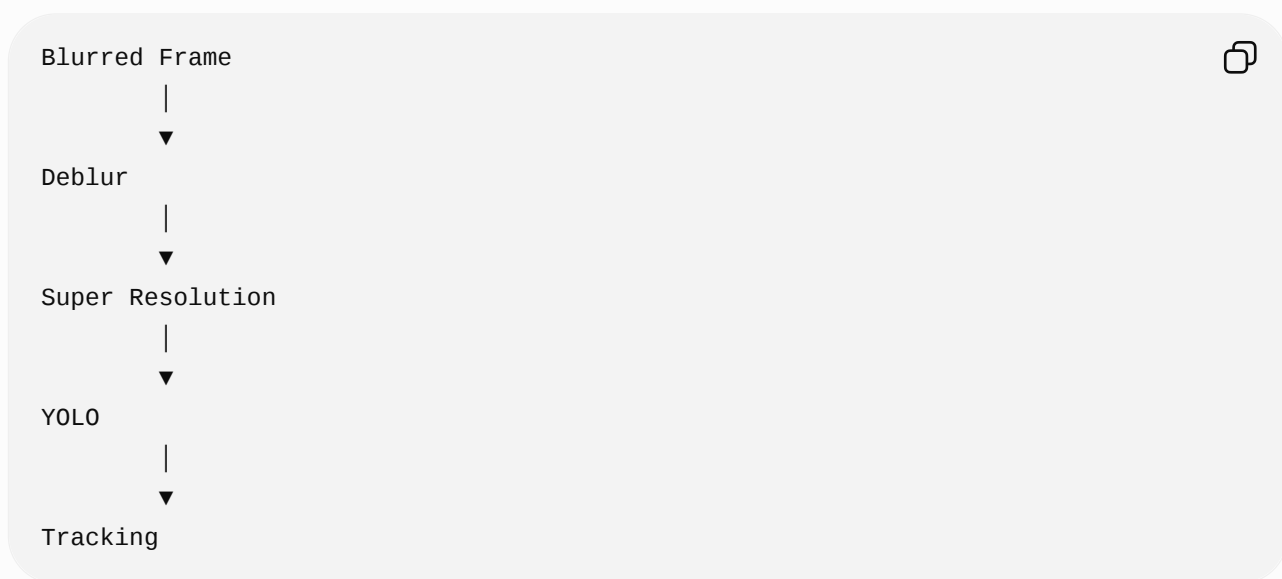
Model	FPS (RTX 3060)
-------	----------------

Model	FPS (RTX 3060)
MIMO-UNet	35–60
NAFNet	25–45
Restormer	10–20
DeblurGAN-v2	20–35
HINet	10–15

For real-time detection, MIMO-UNet or NAFNet are generally better choices.

Step 9: If You Also Want Better Recognition

You can combine restoration models:



This can improve detection of distant or blurry vehicles, but it also increases latency.

Recommendation for Your Project

Since you already have **object detection with tracking IDs**, the most efficient architecture is:

```
Video
|
▼
YOLO Object Detection
|
▼
ByteTrack / DeepSORT
|
▼
For each tracked object:
  Crop object
  ↓
  Deblur (MIMO-UNet or Restormer)
  ↓
  Display or save the enhanced object
```

This avoids deblurring the entire frame, reduces computation, and preserves your existing detection and tracking pipeline. If your goal is to improve detection accuracy on blurry frames (rather than just improving the appearance of tracked objects), move the deblurring stage before the detector instead.



Is this conversation helpful so far?

